

Satellite Building Detection Using YOLOv8

SpaceNet 2 — Paris AOI | Object Detection Project

June 7, 2026

1. Problem

Manually identifying buildings in satellite imagery is slow, expensive, and impossible to scale across the thousands of images captured daily by modern satellites. This project builds an automated pipeline using **YOLOv8** to detect and localize buildings in pan-sharpened RGB satellite crops of Paris, France. The resulting system can support urban planning, disaster damage assessment, and population density estimation at scale.

2. Data

Source: SpaceNet 2 Building Detection Dataset — Paris AOI (AOI_3), available on Kaggle ([ugorjiir/spacenet-2-paris-buildings](https://www.kaggle.com/ugorjiir/spacenet-2-paris-buildings)), originally from DigitalGlobe / CosmiQ Works.

- **Images:** $\approx 1,148$ WorldView-3 pan-sharpened RGB GeoTIFF chips, each 650×650 pixels
- **Buildings:** $\approx 23,816$ annotated footprints (≈ 20 buildings per image on average)
- **Raw labels:** GeoJSON polygon files (geographic coordinates) — converted to YOLO `.txt` bounding boxes using `rasterio` and `shapely`
- **Split:** 80% train (~ 919) / 10% val (~ 115) / 10% test (~ 114), randomised with `train_test_split`

Each YOLO label line encodes one building as:

0 x_c y_c w h (all values normalised to $[0, 1]$)

3. Model

YOLOv8 (Ultralytics, 2023) is a single-stage, anchor-free object detector that processes the full image in one forward pass, predicting bounding boxes and class scores simultaneously. It uses a decoupled detection head and a CSP-based backbone for efficient multi-scale feature extraction.

Property	Value
Variant	YOLOv8s (small)
Parameters	$\approx 11.2\text{M}$
Pre-training	COCO (80 classes, transfer learning)
Input size	640×640 px
Output classes	1 (building)

The small variant was chosen to balance training speed and accuracy given the limited dataset size. Weights from `yolo8s.pt` (COCO pre-trained) were used as the starting point, enabling faster convergence than random initialisation.

4. Training

Parameter	Value
Epochs	50
Batch size	16
Image size	640×640
Optimizer	AdamW
Early stopping patience	10 epochs
GPU	Tesla T4 (Google Colab)
Training time	$\sim X$ min (<i>fill in</i>)

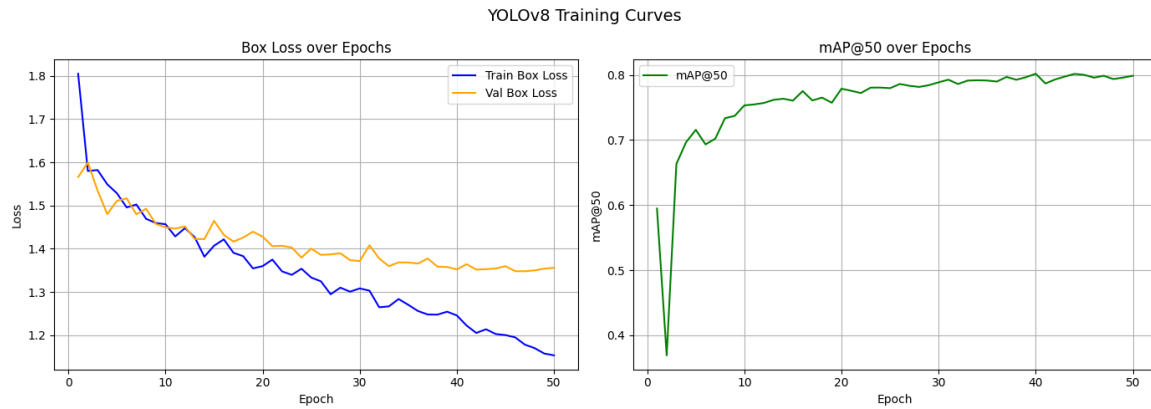


Figure 1: Train vs. Validation Box Loss (left) and mAP@50 (right) over 50 epochs

5. Results

Metric	Value	Plain English
mAP@50	X.XX	Avg. detection quality at $\geq 50\%$ box overlap
Precision	X.XX	Fraction of predicted boxes that are real buildings
Recall	X.XX	Fraction of real buildings the model found
F1 Score	X.XX	Harmonic mean of Precision and Recall

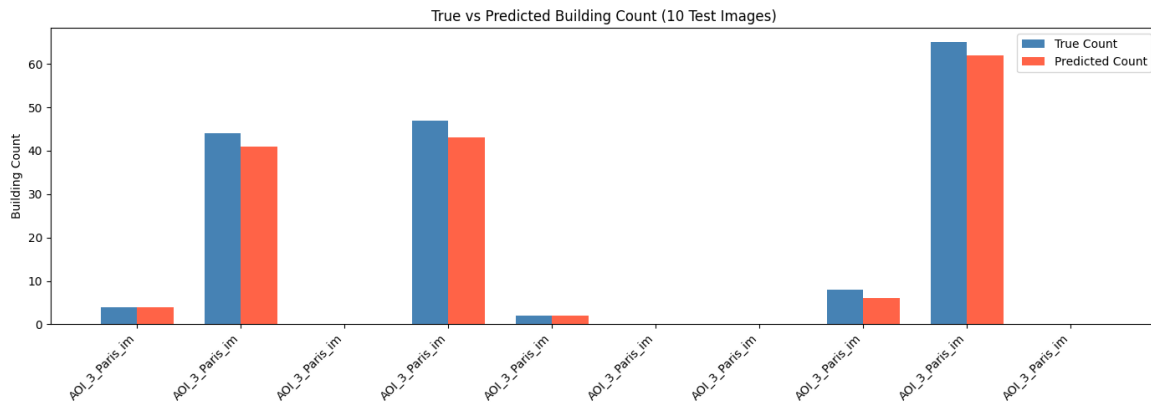


Figure 2: End-to-end pipeline output: original image (left) and YOLOv8 detections with bounding boxes (right)

6. Observations

What worked well:

- Transfer learning from COCO accelerated convergence — useful mAP was reached within the first 15 epochs
- Isolated buildings with clear rooftop boundaries were detected reliably at high confidence
- The GeoJSON $\rightarrow$ YOLO conversion preserved label quality without significant information loss

What the model struggled with:

- Dense clusters where buildings share walls — bounding boxes merged or missed individual structures

- Shadowed rooftops where texture and edges were obscured
- Very small buildings near image borders

One idea to improve: Switching to **YOLOv8m** (~ 25 M parameters) would increase model capacity for dense and small building detection. Alternatively, tiling each 650×650 image into smaller 320×320 overlapping crops before training could significantly improve recall on small structures.